

Sparsity-preserving gradient utility tools for PyTorch

Theodore Barfoot¹, Ben Glocker², and Tom Vercauteren¹

¹ King's College London, London, UK ² Imperial College London, London, UK

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Open Journals](#)

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright
and release the work under a
Creative Commons Attribution 4.0
International License ([CC BY 4.0](#))

Summary

The `torchsparsegradutils` package provides differentiable sparse linear-algebra utilities for PyTorch (Paszke et al., 2019) that preserve sparsity for returned gradients during backpropagation. While PyTorch directly supports sparse tensors, its default semantics treat sparse layouts as storage optimisations rather than a mathematical structure that results in optimising directly for that sparse subspace. Gradients resulting from PyTorch native functions are often dense and incompatible with end-to-end training of models that require fixed sparsity patterns (e.g., sparse covariance/precision structures).

To address this limitation, we introduce `torchsparsegradutils`. Key features include: (1) memory-efficient sparse-dense matrix multiplication with sparse gradient preservation, (2) sparse triangular and generic linear system solvers, enabling sparse gradients during backpropagation, and multiple algorithmic backends (BICGSTAB, CG, LSMR, MINRES), (3) cross-platform sparse solver wrappers for CuPy (Okuta et al., 2017) and JAX (Bradbury et al., 2018), (4) sparse multivariate normal distributions with LL^T and LDL^T sparse covariance and precision matrix parameterisations with reparameterised sampling methods, and (5) specialised encoders for spatial neighbourhood relationships in N-dimensional data.

Statement of need

Many scientific machine learning models benefit from representing large linear operators (e.g., neighbourhood couplings, precision matrices, sparse Jacobians) using sparse tensors to reduce memory and compute complexity. In high-dimensional settings, such as volumetric medical imaging, dense covariance or precision parameterisations are typically intractable, motivating sparse end-to-end parameterisations.

However, learning these models with gradient-based optimisation requires backpropagation through sparse linear algebra (matrix products, triangular solves, and linear system solves). PyTorch's default sparse semantics are not designed to preserve user-imposed sparsity structure during differentiation (PyTorch issue #87448), which can lead to memory blow-ups and prevent end-to-end optimisation of sparse probabilistic models.

`torchsparsegradutils` addresses this gap by implementing custom autograd functions for key sparse operators that return gradients only for stored nonzeros, enabling practical optimisation of models that rely on fixed sparse structure, such as sparse multivariate normal distributions with sparse covariance/precision factors.

State of the field

PyTorch (Paszke et al., 2019) exposes sparse layouts (COO, CSR, and related formats) and implements a growing set of sparse operations. However, PyTorch's design goal is *dense-equivalent semantics* for sparse layouts: a guiding invariant is that applying an operation in

39 sparse form should match applying it in dense form after conversion, including the backward
40 function (PyTorch issue #87448). This makes it difficult to learn parameters that are intended
41 to remain structurally sparse, because gradients may be produced for implicit zeros, or
42 intermediate computations may densify.

43 PyTorch also provides `MaskedTensor`, distinguishing specified and unspecified elements in ten-
44 sors and is conceptually closer to the constrained-subspace interpretation of sparsity. However,
45 `MaskedTensor` remains at prototype stage with incomplete operator coverage, and storing a
46 full boolean mask incurs a significant memory overhead, partially negating the memory benefits
47 of sparse index-based representations for large-scale problems.

48 Other libraries provide efficient sparse kernels but do not directly solve “sparsity-preserving
49 gradients in PyTorch”: SciPy (Virtanen et al., 2020) provides mature sparse linear algebra
50 but no automatic differentiation; CuPy (Okuta et al., 2017) and JAX (Bradbury et al., 2018)
51 provide sparse solvers in their respective ecosystems but are not drop-in components for
52 PyTorch autograd/training loops. GPyTorch (Gardner et al., 2018) targets scalable Gaussian
53 process inference via kernel structure and approximations (e.g., inducing/structured methods)
54 rather than arbitrary user-specified sparse covariance/precision factors. PyTorch Geometric’s
55 `torch_sparse` (Fey & Lenssen, 2019) focuses on graph message-passing primitives rather than
56 sparse covariance/precision modelling and differentiable sparse solves for probabilistic models.

57 Software design

58 `torchsparsegradutils` is built around `torch.autograd.Function` operators that wrap Py-
59 Torch’s forward sparse kernels but override the backward pass to preserve sparsity for selected
60 inputs. This design keeps the user-facing API close to standard PyTorch code while making
61 sparsity preservation an explicit, opt-in choice.

62 Two design trade-offs shaped the implementation. First, the package targets *structure-*
63 *preserving learning* over maximal operator coverage, as only a focused set of operations (sparse
64 matrix products, triangular solves, generic sparse solvers) are implemented, but these are suffi-
65 cient to support sparse multivariate normal sampling and sparse solver-based models. Second,
66 for broad device/backend compatibility, the package combines native PyTorch implementations
67 (iterative Krylov solvers: CG, BiCGSTAB, LSMR, MINRES) with optional wrappers to external
68 libraries (CuPy, JAX), allowing users to trade off portability versus performance.

69 **Build vs. contribute justification.** PyTorch’s current semantics treat sparse layouts as perfor-
70 mance optimisations and prioritise the dense-equivalence invariant (PyTorch issue #87448). In
71 contrast, this package intentionally provides *structure-preserving* backward passes for specific
72 operators to enable learning with fixed sparsity patterns (e.g., sparse triangular factors for
73 covariance/precision). This difference is semantic (not just implementation), so the function-
74 ality is better delivered as an opt-in external library rather than changing PyTorch’s default
75 behaviour.

76 Research impact statement

77 This software provides an opt-in path to sparsity-preserving gradients for sparse linear algebra
78 in PyTorch, enabling research prototypes that would otherwise be limited by dense gradients
79 or densification. The package is currently being used in active research projects for medical
80 image segmentation, though publications resulting from this work are still in preparation.

81 The codebase demonstrates community-readiness through comprehensive infrastructure: docu-
82 mentation with quickstart guides and API references, extensive test coverage across all modules,
83 CI/CD pipelines for automated testing, and an open contribution process via GitHub issues
84 and pull requests. The codebase has been developed openly over multiple years with public

85 commit history, releases, and issue tracking. Benchmark suites comparing solver performance
86 across problem sizes and sparsity patterns provide reproducible reference materials.

87 Given the broad applicability of sparse structured Gaussians—spanning medical imaging,
88 spatial statistics, geostatistics, and large-scale probabilistic modelling, we anticipate growing
89 adoption as the research community increasingly requires memory-efficient optimisation of
90 high-dimensional probabilistic models.

91 Mathematics

92 Sparse Matrix Operations

93 Sparse Matrix Multiplication

94 The package implements sparse-dense matrix multiplication $\mathbf{C} = \mathbf{AB}$ where $\mathbf{A} \in \mathbb{R}^{m \times n}$ is
95 sparse and $\mathbf{B} \in \mathbb{R}^{n \times p}$ is dense. The forward pass uses PyTorch's native `torch.sparse.mm`,
96 while the backward pass is reimplemented to preserve sparsity patterns in the gradients.

97 Given upstream gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{C}} \in \mathbb{R}^{m \times p}$ from some scalar objective function $\mathcal{L}$, the chain rule
98 gives:

99 **Gradient wrt $\mathbf{B}$ (dense):**

$$\frac{\partial \mathcal{L}}{\partial \mathbf{B}} = \mathbf{A}^\top \frac{\partial \mathcal{L}}{\partial \mathbf{C}}.$$

100 **Gradient wrt $\mathbf{A}$ (sparse):** For a nonzero entry $\mathbf{A}_{ij}$,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{A}_{ij}} = \sum_{k=1}^p \left(\frac{\partial \mathcal{L}}{\partial \mathbf{C}_{ik}} \right) \mathbf{B}_{jk}.$$

101 Sparse Linear System Solvers

102 The package provides multiple approaches for solving sparse linear systems

$$\mathbf{Ax} = \mathbf{B},$$

103 where $\mathbf{A} \in \mathbb{R}^{n \times n}$ is sparse, $\mathbf{B} \in \mathbb{R}^{n \times p}$ is dense (with $p = 1$ for a single right-hand side), and
104 $\mathbf{x} \in \mathbb{R}^{n \times p}$ is the dense solution. We support both direct triangular solves and iterative solvers
105 (CG, BiCGSTAB, LSMR, MINRES). All are differentiable via the implicit function theorem.

106 Given upstream gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{x}} \in \mathbb{R}^{n \times p}$:

107 **Gradient wrt $\mathbf{b}$ (dense):**

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \mathbf{A}^{-\top} \frac{\partial \mathcal{L}}{\partial \mathbf{x}}.$$

108 **Gradient wrt $\mathbf{A}$ (sparse):** The dense form would be

$$\frac{\partial \mathcal{L}}{\partial \mathbf{A}} = - \left(\mathbf{A}^{-\top} \frac{\partial \mathcal{L}}{\partial \mathbf{x}} \right) \mathbf{x}^\top.$$

109 For a nonzero entry $\mathbf{A}_{ij}$ this becomes

$$\frac{\partial \mathcal{L}}{\partial \mathbf{A}_{ij}} = - \sum_{k=1}^p \left(\mathbf{A}^{-\top} \frac{\partial \mathcal{L}}{\partial \mathbf{x}} \right)_{ik} \mathbf{x}_{jk}$$

110 Sparse Multivariate Normal Distributions

111 The package implements sparse multivariate normal distributions $\eta \sim \mathcal{N}(\mu, \Sigma) \equiv \mathcal{N}(\mu, \Omega^{-1})$
 112 with two parameterisations for efficient sampling. Both methods transform standard normal
 113 samples $\epsilon \sim \mathcal{N}(0, I)$ into samples from the desired multivariate normal distribution:

114 **$LL^\top$ Parameterisation:** Sparse lower triangular matrices L with positive diagonals:

$$\eta_\Sigma = \mu + L_\Sigma \epsilon, \quad \eta_\Omega = \mu + L_\Omega^{-\top} \epsilon$$

115 **$LDL^\top$ Parameterisation:** Sparse unit lower triangular matrices L and diagonal D :

$$\eta_\Sigma = \mu + LD^{1/2} \epsilon, \quad \eta_\Omega = \mu + L_\Omega^{-\top} D_\Omega^{-1/2} \epsilon$$

116 We store L without its diagonal (strictly lower), and treat it as unit lower-triangular at use
 117 time. The $LDL^\top$ parameterisation provides superior numerical stability for precision matrices
 118 by avoiding strict positive definiteness constraints.

119 AI usage disclosure

120 Generative AI tools were used during development of this software and manuscript. Various large
 121 language models were used to assist with code generation, refactoring, and test scaffolding
 122 for portions of the codebase, and AI assistance was used to draft and edit parts of the
 123 documentation and this manuscript. The repository was initiated prior to widespread AI coding
 124 assistant adoption, with AI tools incorporated during later development phases. All AI-assisted
 125 outputs were reviewed, edited, and validated by the human authors, who take responsibility for
 126 the final software and paper.

127 Acknowledgements

128 We thank the PyTorch development team for foundational sparse tensor support. We also
 129 acknowledge upstream solver implementations and references used as starting points for iterative
 130 methods (pykrylov, cornellius-gp/linear_operator, pytorch-minimize) (Saad, 2003). We thank
 131 Floris Laporte for his excellent tutorial on implementing sparse linear system solvers in PyTorch
 132 (Laporte, 2020), which provided valuable insights for gradient computation strategies. This
 133 work was supported by the Wellcome/EPSRC Centre for Interventional and Surgical Sciences
 134 and the School of Biomedical Engineering & Imaging Sciences, King's College London.

135 References

- 136 Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G.,
 137 Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). *JAX: Composable*
 138 *transformations of Python+NumPy programs*. <https://github.com/jax-ml/jax>
- 139 Fey, M., & Lenssen, J. E. (2019). Fast graph representation learning with PyTorch Geometric.
 140 *ICLR Workshop on Representation Learning on Graphs and Manifolds*. <https://arxiv.org/abs/1903.02428>
- 142 Gardner, J. R., Pleiss, G., Bindel, D., Weinberger, K. Q., & Wilson, A. G. (2018). GPyTorch:
 143 Blackbox matrix-matrix gaussian process inference with GPU acceleration. *Advances in*
 144 *Neural Information Processing Systems*, 31. <https://doi.org/10.5555/3327757.3327857>
- 145 Laporte, F. (2020). *Solving sparse linear systems in PyTorch*. <https://blog.flaport.net/solving-sparse-linear-systems-in-pytorch.html>.

- 147 Okuta, R., Unno, Y., Nishino, D., Hido, S., & Loomis, C. (2017). CuPy: A NumPy-compatible
148 library for NVIDIA GPU calculations. *Proceedings of the Workshop on Machine Learning*
149 *Systems (LearningSys) at the 31st Conference on Neural Information Processing Systems*,
150 1–7. https://learningsys.org/nips17/assets/papers/paper_16.pdf
- 151 Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T.,
152 Lin, Z., Gimelshein, N., Antiga, L., & others. (2019). PyTorch: An imper-
153 ative style, high-performance deep learning library. *Advances in Neural Infor-*
154 *mation Processing Systems*, 32, 8024–8035. [https://papers.neurips.cc/paper/](https://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library)
155 [9015-pytorch-an-imperative-style-high-performance-deep-learning-library](https://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library)
- 156 Saad, Y. (2003). *Iterative methods for sparse linear systems* (2nd ed.). Society for Industrial;
157 Applied Mathematics. <https://doi.org/10.1137/1.9780898718003>
- 158 Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D.,
159 Burovski, E., Peterson, P., Weckesser, W., Bright, J., & others. (2020). SciPy 1.0:
160 Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3),
161 261–272. <https://doi.org/10.1038/s41592-019-0686-2>

DRAFT